EX16b - Practica de mascaras - AND, OR, NOT, any y all

Guia paso a paso para principiantes

Esta guia sigue el archivo real `ex16b_masks_filtering_practice.py`. Se explica primero el problema y despues el codigo desde el punto de vista de alguien que esta empezando Python.

1. Que queremos resolver

Practicar filtrado sobre un dataset [time, altitude, velocity, temperature] usando mascaras simples y compuestas.

Variables / elementos del problema

Nombre	Significado
sensor_data	matriz 9x4
mask	condicion por fila

Ecuacion o logica en papel

```
mask = condition
values = vector[mask]
rows = matrix[mask]
```

Comprobacion manual

altitudes>=100 es True en indices [3,4,5,6], por lo que hay cuatro muestras de alta altitud.

2. Lo que ya sabemos y lo nuevo

Ya visto:

- mascaras
- A[mask]
- np.where
- np.diff

Nuevo en este ejercicio:

- extraer columnas
- x[mask] vs A[mask]
- |
- ~
- np.min
- np.abs
- np.any
- np.all

3. Conceptos Python nuevos

1. Valores vs filas

Sintaxis: `altitudes[mask] vs sensor_data[mask]`

Sobre un vector obtenemos valores; sobre la matriz completa obtenemos filas.

2. |

Sintaxis: `mask_a | mask_b`

True si al menos una de las condiciones es True en esa posicion.

3. ~

Sintaxis: `~mask`

Invierte cada booleano. ~(velocities>0) incluye tambien velocidad cero.

4. np.any

Sintaxis: `np.any(mask)`

True si existe al menos un True.

5. np.all

Sintaxis: `np.all(mask)`

True solo si todos los valores son True.

4. Codigo paso a paso

Paso 1. Extraer columnas

```
times = sensor_data[:, 0]
altitudes = sensor_data[:, 1]
velocities = sensor_data[:, 2]
temperatures = sensor_data[:, 3]
```

Nombrar columnas hace las expresiones posteriores mas legibles.

Paso 2. Mascaras simples

```
altitude_equal_above_100 = altitudes >= 100
velocity_positive = velocities > 0
```

Cada resultado es un vector de nueve booleanos.

Paso 3. Valores y filas

```
high_altitude_values = altitudes[altitude_equal_above_100]
high_altitude_states = sensor_data[altitude_equal_above_100]
```

La misma mascara selecciona cosas distintas segun la estructura a la que se aplique.

Paso 4. AND y OR

```
(altitudes >= 100) & (velocities > 0)
(altitudes >= 100) | (temperatures < 12)
```

& exige ambas condiciones; | exige al menos una.

Paso 5. NOT

```
not_ascending = ~(velocities > 0)
```

Esto representa no positivo, es decir, <=0; no es exactamente lo mismo que velocities<0.

Paso 6. Variable derivada

```
speed = np.abs(velocities)
processed_data = np.column_stack((sensor_data, speed))
```

np.abs elimina el signo y crea rapidez; column_stack la agrega al dataset.

Paso 7. Preguntas globales

```
np.any(altitudes > 130)
np.all(temperatures > 0)
```

Estas funciones reducen una mascara completa a una sola respuesta booleana.

5. Flujo de ejecucion

```
sensor matrix
 -> named columns
 -> simple masks
 -> values/rows
 -> AND/OR/NOT
 -> statistics / derived column
 -> any/all
```

6. Resultado / comprobacion esperada

El archivo encuentra 4 muestras con altitude>=100, 5 con velocity>0, 4 con velocity<0 y devuelve YES para any altitude>130 y all temperature>0.

7. Errores tipicos al empezar

- Interpretar ~(v>0) como estrictamente v<0.
- Usar & o | sin parentesis.
- Confundir valores con filas.
- Confundir np.any con np.all.

8. Si solo recuerdas cinco cosas

- Un mismo mask puede filtrar un vector o una matriz.
- &=AND, |=OR, ~=NOT.
- np.abs convierte velocidad firmada en rapidez.
- np.any pregunta si existe alguno.
- np.all pregunta si se cumplen todos.

9. Mini practica

Cambia un unico input. Antes de ejecutar, escribe en un comentario que esperas que cambie y por que. Despues ejecuta y compara la prediccion con la salida.

Criterio de cierre

Puedes continuar cuando puedas explicar el problema, la ecuacion/logica, los conceptos Python nuevos y el flujo de ejecucion sin limitarte a leer el codigo.